ScopePilot — Work Summary

Part 1: Codebase Analysis, Architecture Discovery & Responsibilities

MODULE:	OWNER:	DATE:	STATUS:
User & Workspace Mgmt	Sushant	October 11, 2026	Analysis Complete

01 Executive Summary & Project Overview

ScopePilot is an AI-powered freelance management platform designed to combat scope creep. Freelancers and clients manage agreed scopes, proposals, milestones, deliverables, and change requests in one centralized system. Today, we completed the full architectural investigation of the existing backend codebase to establish Sushant's operational boundaries for the User & Workspace Management module before writing any code.

02 Existing Backend Architecture & Request Flow

The backend is built with Node.js, Express, MongoDB (Mongoose), and Zod. It follows a strict 3-tier modular architecture across all features (src/modules/<moduleName>/):

1. **HTTP Request** — Triggered from frontend or testing client (e.g. POST /api/workspaces)
2. **Route Index (routes.js)** — Directs the request to the workspace route mount under /api
3. **Auth Guard (auth.middleware.js)** — Verifies the Bearer JWT token and attaches authenticated user to req.user
4. **Zod Validation (validate.middleware.js)** — Validates body/params against schema before business logic executes
5. **Controller (workspace.controller.js)** — Extracts inputs and returns standard ApiResponse envelopes
6. **Service (workspace.service.js)** — Executes core business rules, queries, and permission validations
7. **Model (workspace.model.js)** — Persists and retrieves workspace documents from MongoDB

03 Files Examined & Module Ownership

Audited inventory of relevant files in the ScopePilot repository:

[OWNED] backend/src/modules/workspace/workspace.model.js
Workspace schema with owner, members array, and invitations array

[OWNED] backend/src/modules/workspace/workspace.service.js
Business operations for creation, retrieval, and invitation management

[OWNED] backend/src/modules/workspace/workspace.controller.js
HTTP handlers wrapping service calls with ApiResponse formatting

[OWNED] backend/src/modules/workspace/workspace.validation.js
Zod schemas validating workspace name, IDs, and invitation emails

[OWNED] backend/src/modules/workspace/workspace.routes.js
Express route declarations protected by requireAuth

[OWNED] backend/src/modules/workspace/workspace.constants.js
Roles (owner, admin, member, client) and invitation statuses

[REFERENCE] backend/src/modules/auth/auth.model.js
Nikhil's User model. Referenced by workspace schema (ref: 'User')

[COORDINATE] backend/src/middleware/auth.middleware.js
Shared JWT authentication middleware providing req.user

[CONSUMER] backend/src/modules/project/project.model.js
Abhtej's Project model linking directly to Workspace (ref: 'Workspace')

04 Scope Boundaries: What Sushant Owns vs Out-of-Scope

SUSHANT BUILDS (IN SCOPE)

- Workspace creation & setting creator as owner
- Workspace retrieval by ID with auth protection
- Workspace membership tracking & duplicate prevention
- Member listing endpoint for collaboration views
- Workspace invitation issuance & status tracking
- Workspace access authorization helper

DO NOT BUILD (OUT OF SCOPE)

- User registration / login / passwords (Nikhil)
- Projects & proposals business logic (Abhtej)
- Milestones & deliverables tracking (Mounish)
- Change request scope comparisons (Mokshagna)
- AI agents, RAG, Ollama vector search (src/ai)
- Frontend web client UI (Taran)

05 Team Coordination Matrix

- **Nikhil (Auth & Security):** Coordinate on req.user payload, session token lifecycle, and role checks.
- **Abhtej (Projects & Proposals):** Supply the Workspace ID/project relationship needed by Project models.
- **Taran (Frontend Client):** Provide final request/response JSON contracts for workspace & member views.
- **Bharat (QA & Testing):** Share endpoints, test scenarios, and expected response payloads for testing.
- **Shiva (Git & Merging Lead):** Develop strictly on feature/workspace and submit PR for final review.
- **Vivek (Project Lead):** Notify once workspace APIs are tested and ready for end-to-end integration.

06 Prerequisites & Missing Items Identified

[!] Environment File (.env): Only .env.example exists currently. We must copy .env.example to .env so MONGODB_URI and JWT_SECRET are loaded.
[!] MongoDB Daemon: Ensure MongoDB is running locally on port 27017 (or configure a valid MongoDB Atlas URI).
[!] Git Feature Branch: Working tree is currently on main. We must create and switch to feature/workspace before writing code.

07 Step-by-Step Roadmap to Part 2

'Step 1: Switch to feature/workspace branch and setup .env configuration.
'Step 2: Audit workspace.model.js to ensure indexes and subdocuments strictly match requirements.
'Step 3: Refine workspace.service.js to enforce access authorization and prevent duplicate invites.
'Step 4: Execute database seed (npm run seed) and verify all 4 endpoints (POST, GET by ID, GET members, POST invites).
'Step 5: Hand off JSON contracts to Taran and submit clean Pull Request to Shiva.